2026

User-Documentation

JUSTIN LEYSER

MATRIKELNR.: 305941

Inhalt

1. Start here 3
How to make a “Hello world”: 3
Step 1: We import the asset 3
Step 2: We set the scene 3
Step 3: Prepare the Event system: 4
Step 4: We configure the Button or Script that should call the dialog 4
Step 5: Create a Dialogue 5
Step 6: Edit the Dialogue 6
Step 8: Setting up the Dialog 8
Step 9: We test 9
2. How it works 9
3. Components and objects 11
DialogManager 11
Dialog List 11
Animation List 12
Unity Event List 13
BackGroundManager 13
SFX Manager 14
Dialog 15
Actions 16
ActionDialog 16
ActionQuestionWithInput 17
ActionBranchDialog 18
ActionEndDialog 20
ActionStartOtherDialog 20
ActionSceneLoad 20
ActionTriggerAnimation 21
ActionPlaySFX 21
ActionChangeBackground 21
ActionEvent 22
ActionWait 22
How to create your own custom Action 22

4. How to use variables 24
5. How to change the UI of the Dialog 25

Original Unity version: Unity 6 (6000.2.8f1)

(It is perfectly compatible with previous versions)

Required packages:

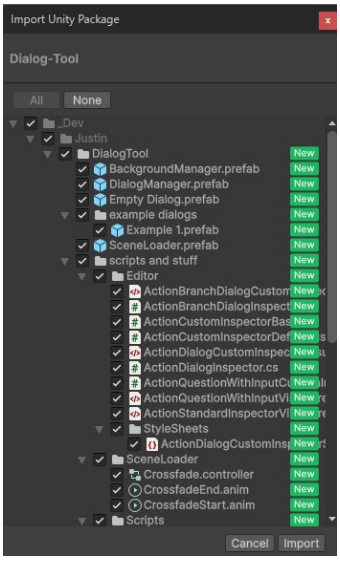
- **TextMesh Pro**

1. Start here

How to make a “Hello world”:

“Hello world” is a phrase used in programming to show on the screen that a program is running correctly. In our case, we are going to have a character say hello world.

Step 1: We import the asset



Make sure the following packages are installed beforehand:

- TextMesh Pro

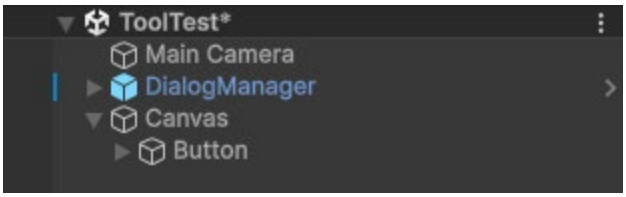
Step 2: We set the scene

The scene must have:

- DialogManager-Prefab
- some button or self-written script that starts the Dialog

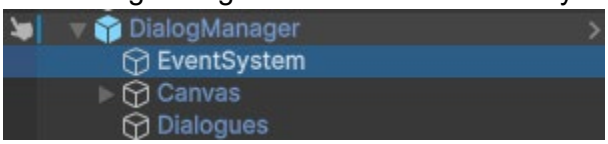
You can add optionally:

- BackgroundManager if you want to change backgrounds
- SceneLoader if you want to switch scenes
- SFXManager if you want to play soundeffects

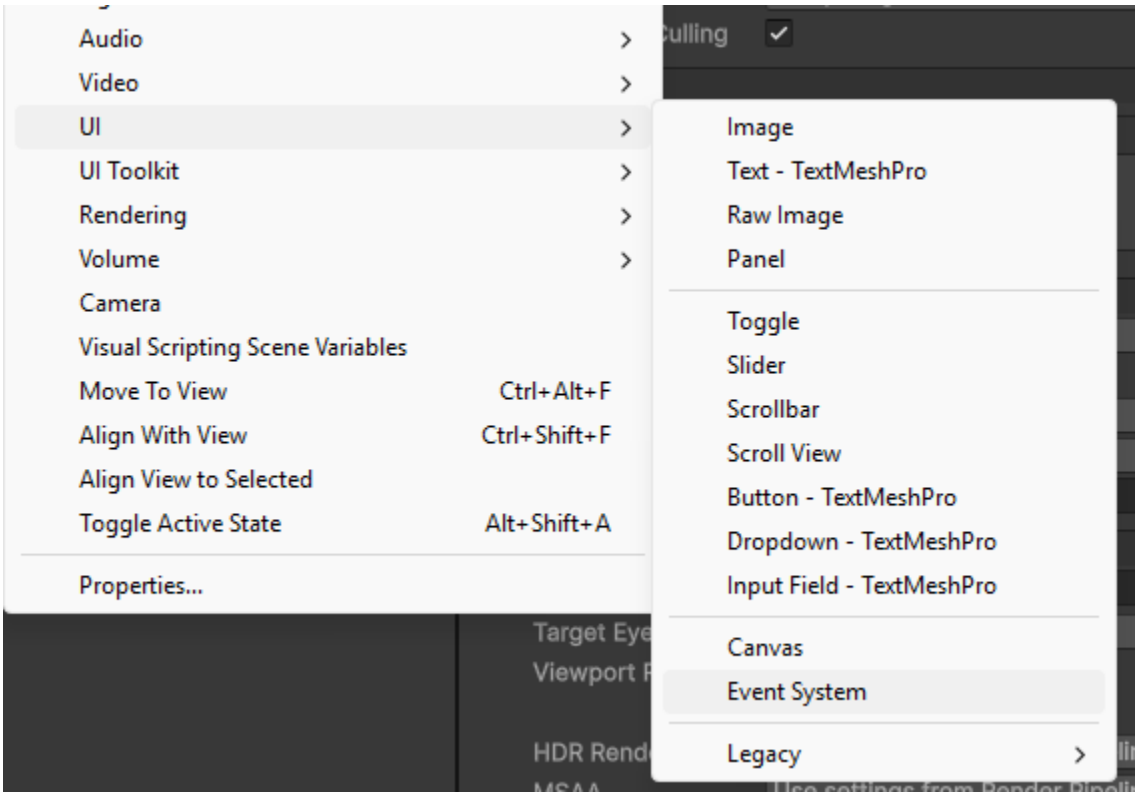


Step 3: Prepare the Event system:

The DialogManager comes with an event system.



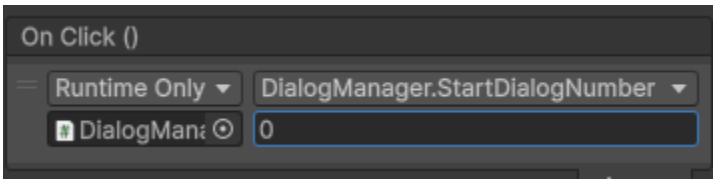
If it’s not compatible with your Unity version you can create a new one and delete the old one. **For this we right click UI/ Event System.**

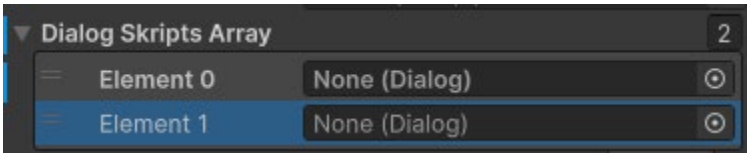


This completes the UI setup.

Step 4: We configure the Button or Script that should call the dialog

For this, we select the button and attach the DialogManager to its OnClick()-Function. Then we select the StartDialogNumber(int) function to be called. The DialogManager contains a list of all dialogs for the scene so with the index you can choose which dialog in the list will be played. For now, leave we leave it at 0.





If you want to use your own scripts to trigger the dialogues you can do this within your scripts by easily calling the public StartDialogNumber(int) function from there. The DialogManager is a singleton, so you can call it like this:

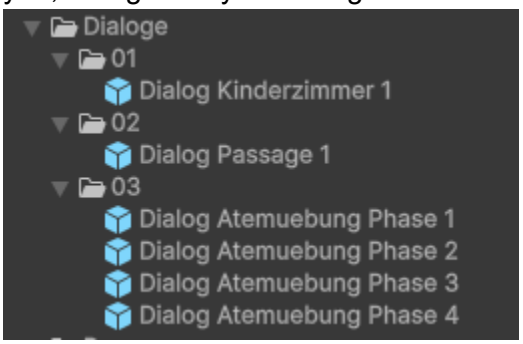
```
Unity-Nachricht | 0 Verweise
private void Start()
{
    DialogManager.Instance.StartDialogNumber(0);
}
```

Remember to pass the index of the dialog in the DialogManager list.

Step 5: Create a Dialogue

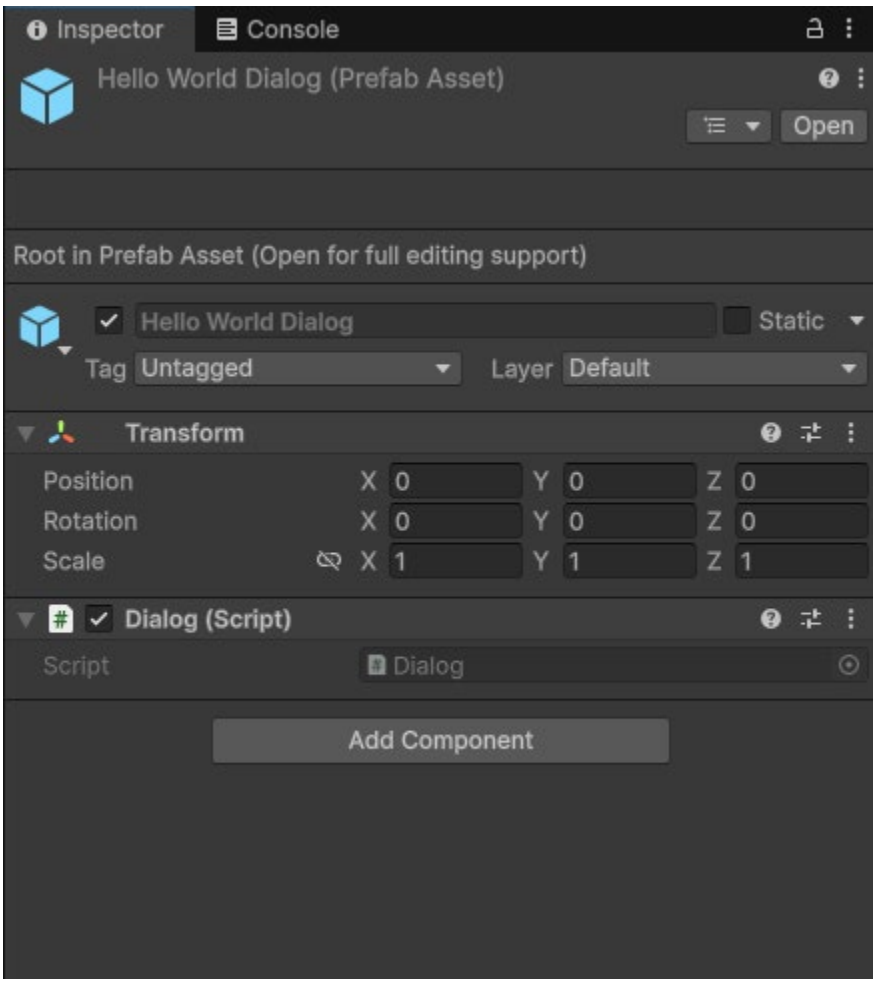
We duplicate the EmptyDialog prefab in the Project Window and rename it to something like “Hello World Dialog”

You don’t have to name the dialogues, but I recommend it cause otherwise you will lose the overview over all your dialogues at some point in the future. For this purpose, I also recommend you, to organize your dialogues in a structured folder hierarchy like this:

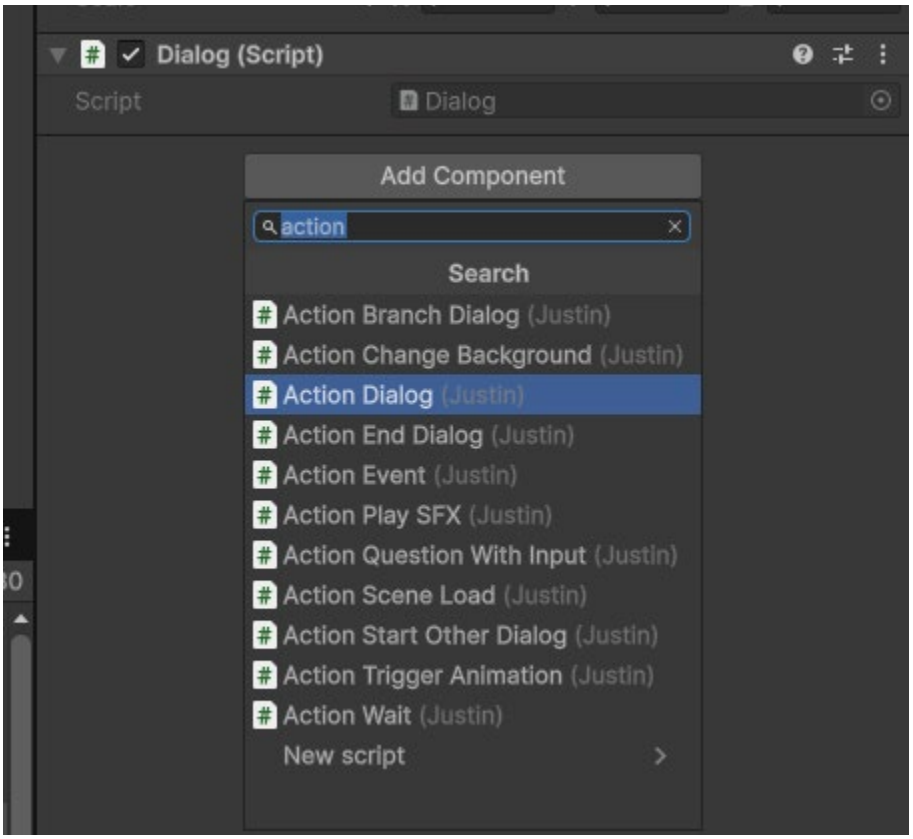


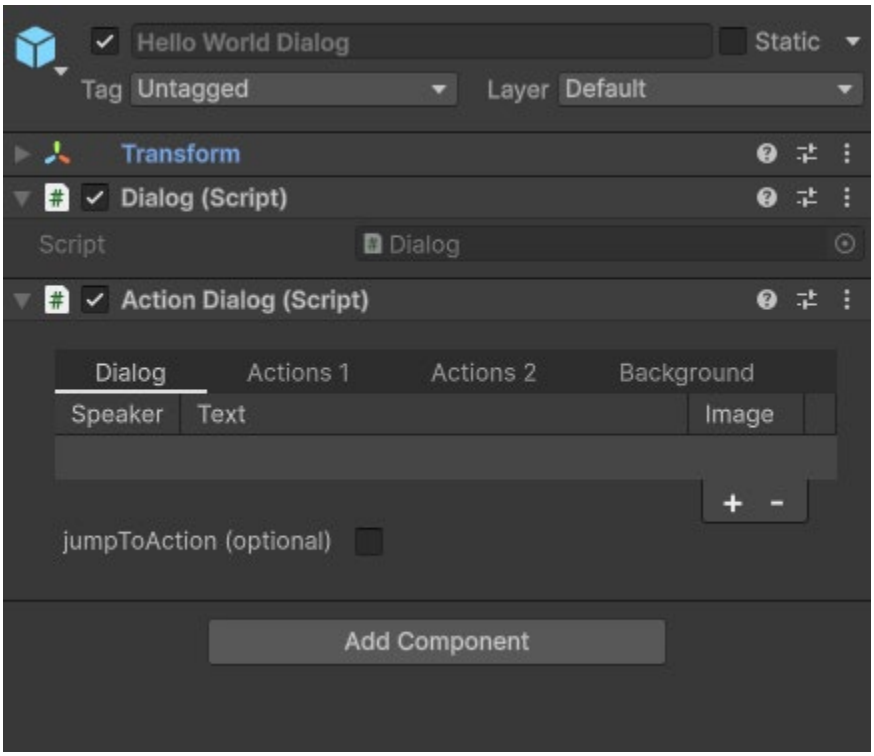
Step 6: Edit the Dialogue

Once the Dialog Prefab is created, we click on it. The Inspector will show an empty dialog with just the dialog script attached to it. This script is necessary for handling the dialogue, so you shouldn't remove it.

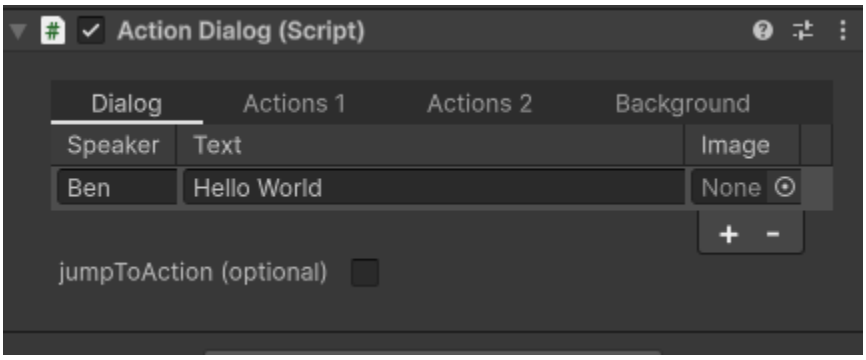


Now, to build the dialog, we need to press "Add Component" and search for the keyword "Action". Every script with "Action" in its name is a possible building block. In our case we just want to display normal dialogue, so we attach the "Action Dialog" script.



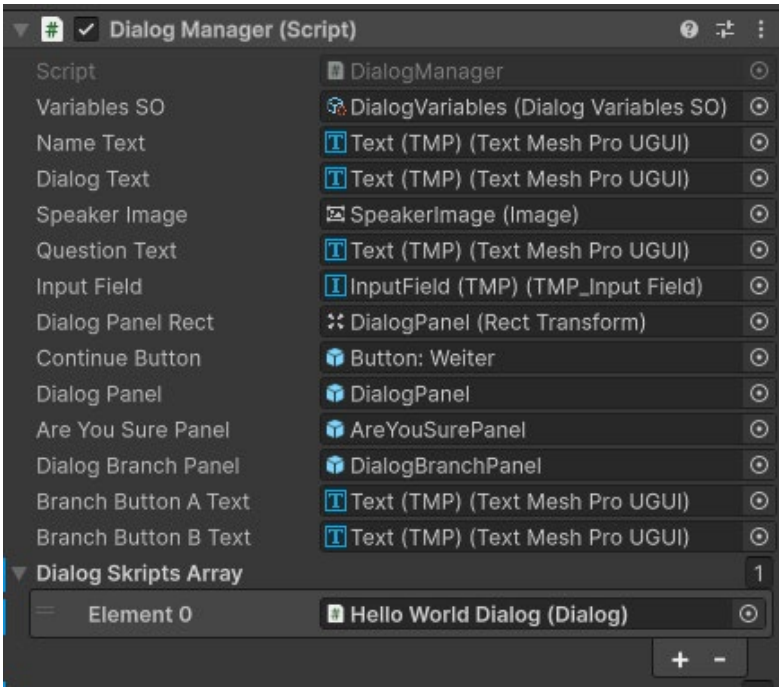


Now we can click the + to add one line to the dialog. In the speaker column we put the name of the speaker and in the text column we write “Hello World”



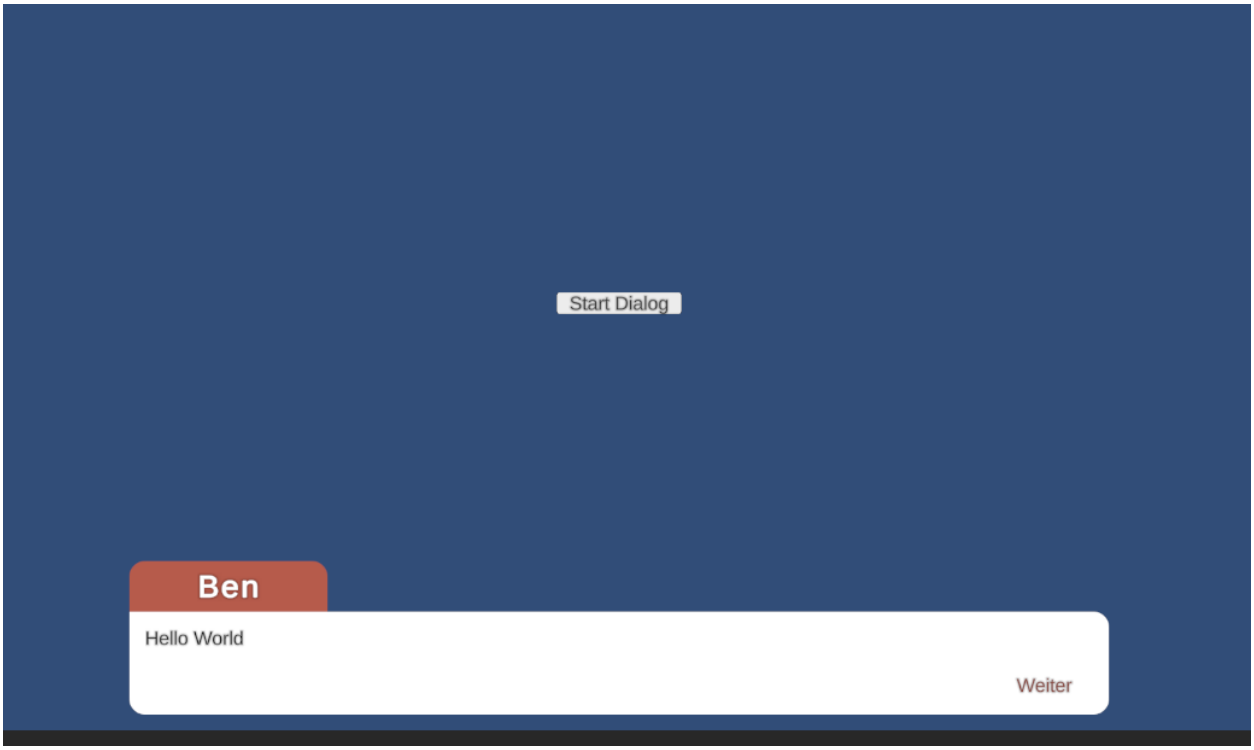
Step 8: Setting up the Dialog

Lastly, we need to drag the “Hello World” Dialog Prefab from the Project Hierarchy into the Dialog List of the DialogManager in the scene. Drag it at index 0.



Step 9: We test

Now we can start play mode and press the earlier created button to test.



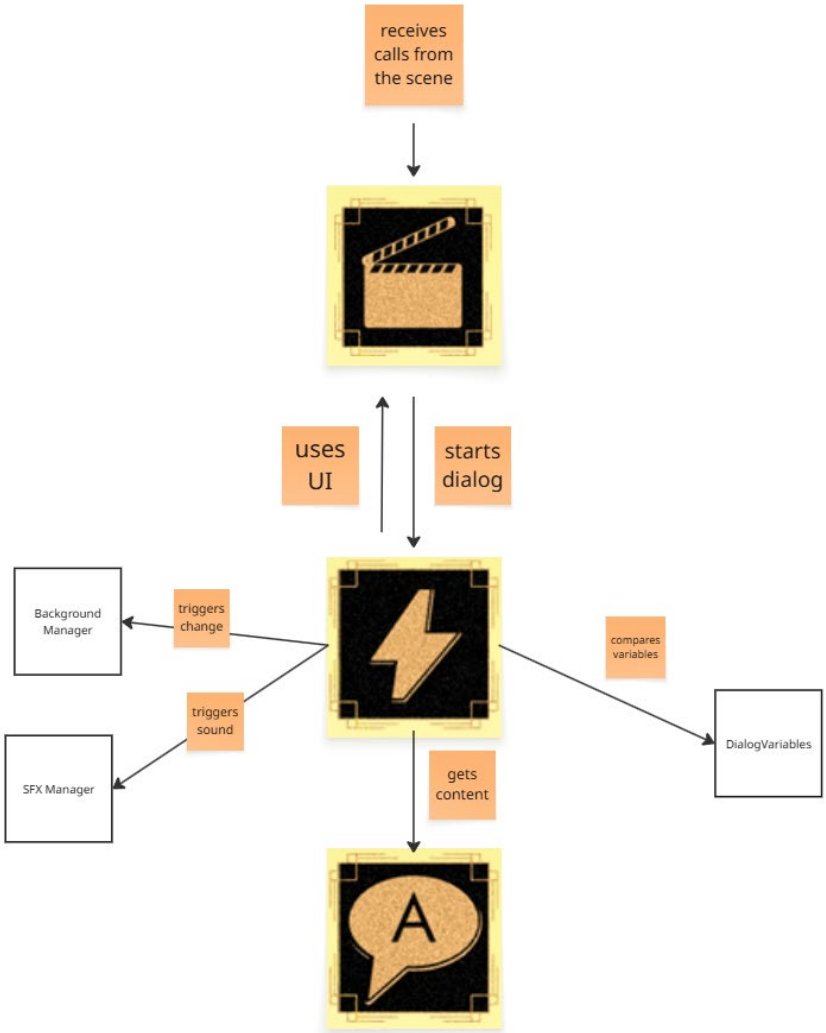
Success!

2. How it works

Dialogue Manager: This is responsible for listening to start dialog calls of your Gameworld and handling the UI for the dialogs. Its role is to receive information which dialog should be started and to tell this dialog that he should play. It is as if an assistant movie director receives orders which actor (dialog) should play in the scene und tells the actor afterwards to start playing their role.

Dialog: This is responsible for executing the dialog. Its role is to read and execute dialog actions based on attached ActionScripts. It is as if an actor plays his role in the scene according to his script.

Action Script: This is responsible for representing the content of a dialog, so the DialogScript knows what to do in which order. Its role is to build the script of a dialog together with other action scripts on a dialog prefab.



BackgroundManager: This is responsible for manipulating the background. Its role is to receive a 2D-sprite and change the background to it.

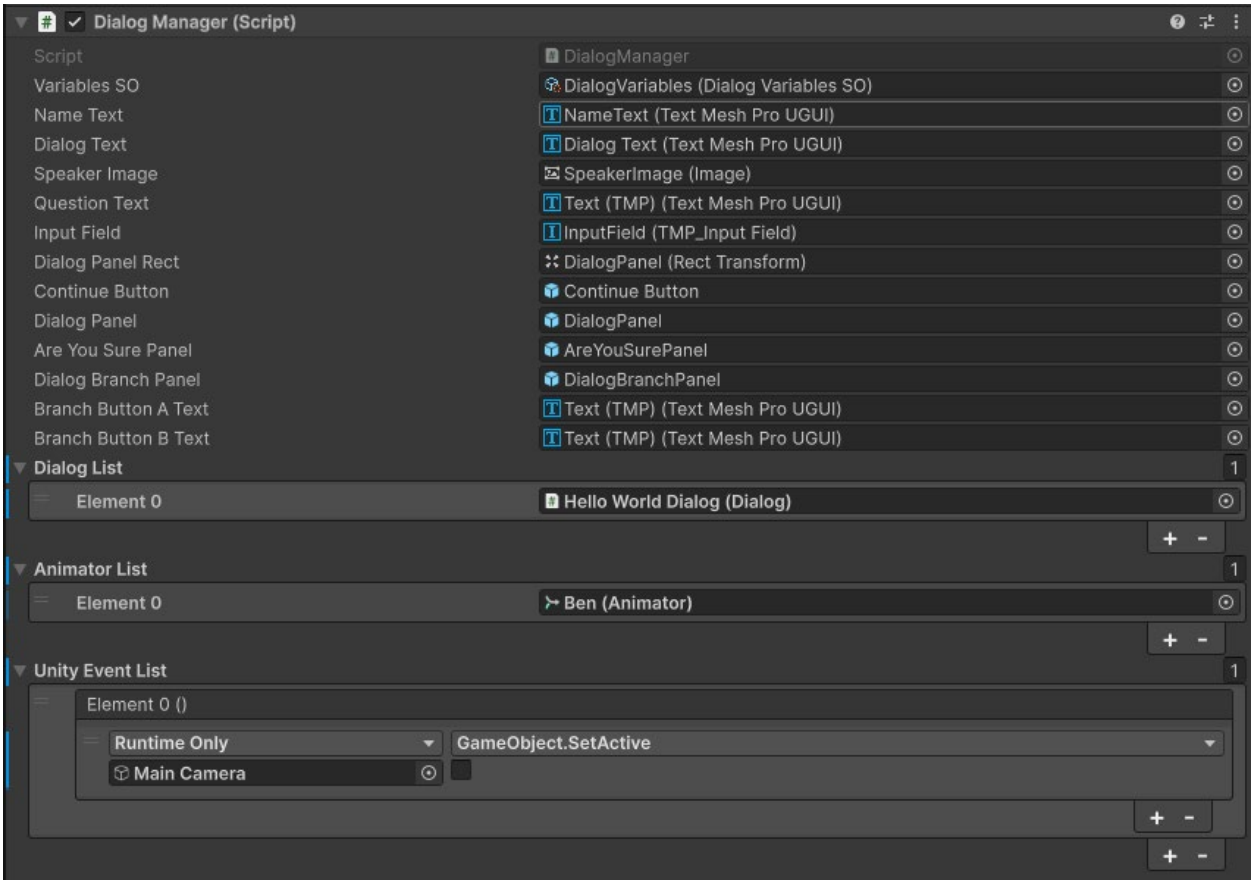
SFX-Manager: This is responsible for playing sound effects. Its role is to receive an audio clip and play it.

Completely independently there is the “DialogVariables”- Scriptable Object. This is responsible for creating and saving variables that can be used in the dialogues.

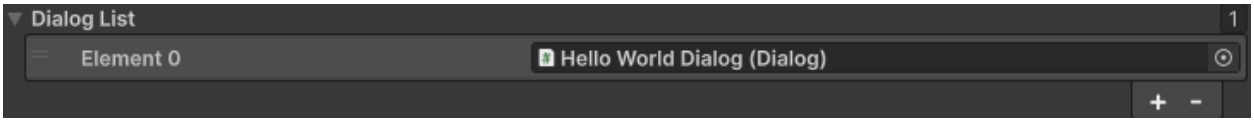
3. Components and objects

DialogManager

This component is responsible for executing the dialogue.



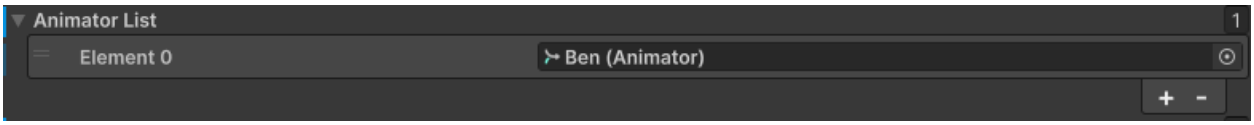
Dialog List



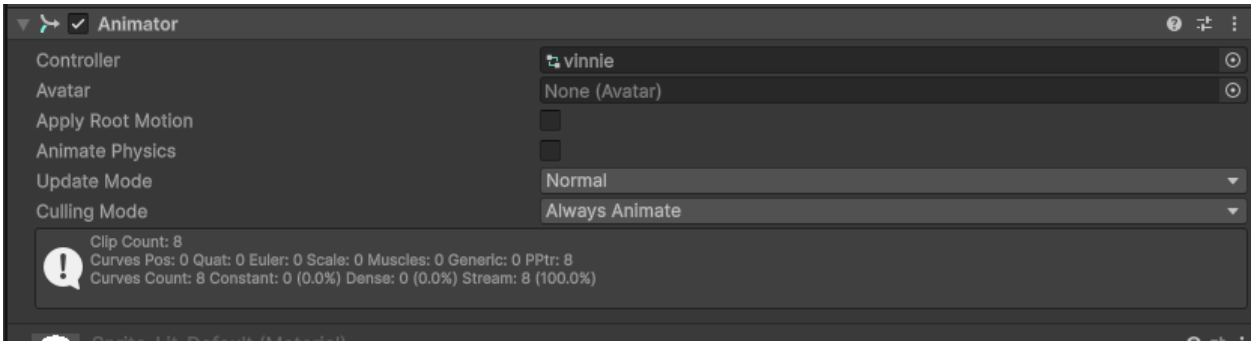
A List that should contain all dialogs you want to play in the scene. It **cannot** be left empty. You need to fill it with Dialog-Prefabs from the Project-Window. There should never be dialogs in the scene.

You can start dialogs of the List by calling the public function StartDialogNumber(int) in the DialogManager script. You can do this by Script or Unity Event. The int represents the position of the script in the list. The list starts with Position 0.

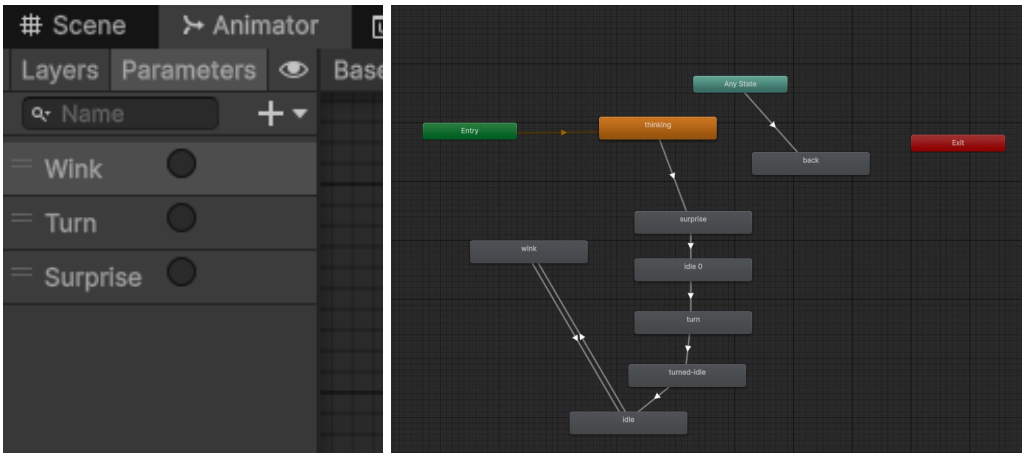
Animation List



This List should contain all animators of the scene that you want to trigger. However it can be left empty. You can drag gameobjects with an animator attached to it into this list and Unity will find the animator automatically.



You can only use gameobjects that have an “animator” component attached to it, which again is referencing an “animator controller”. You also have to create and build the animator controllers yourself and define triggers in them:



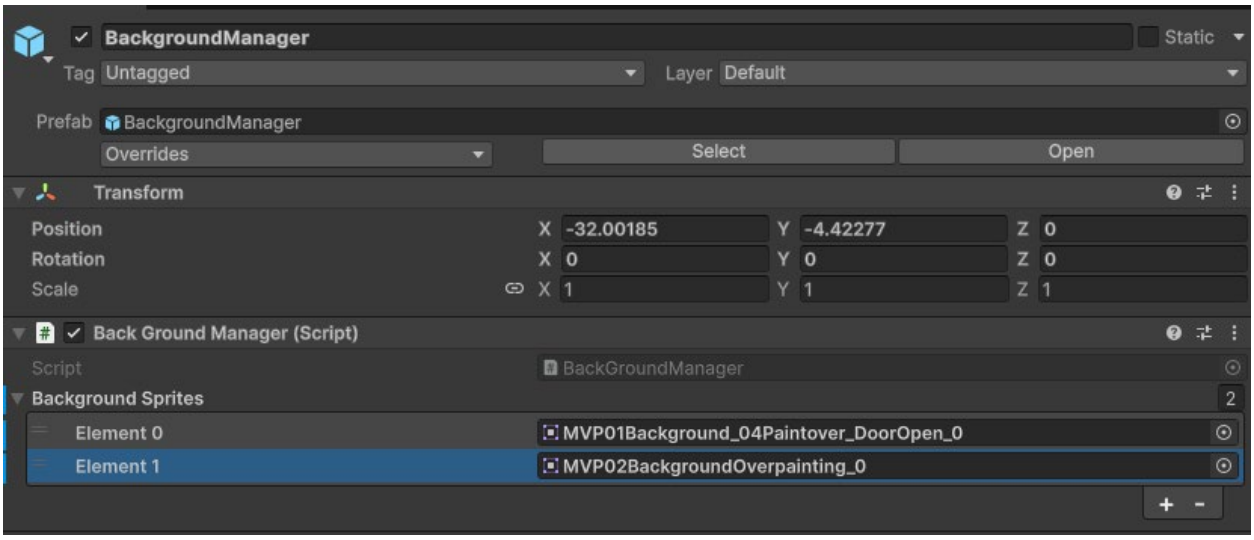
The gameobjects and triggers **must** be named unique!

Unity Event List

This List should contain all Unity Events you want to execute in this scene. However it can be left empty. The events can be called with the action scripts by index. The index represents the position in this list. The list starts with Position 0. If there are multiple events at one position, all the events at this position will be called at the same time.

BackGroundManager

This Manager needs to exist in the scene so you can change the background with the tool. For that you can just drag and drop the prefab from the project window in the hierarchy.

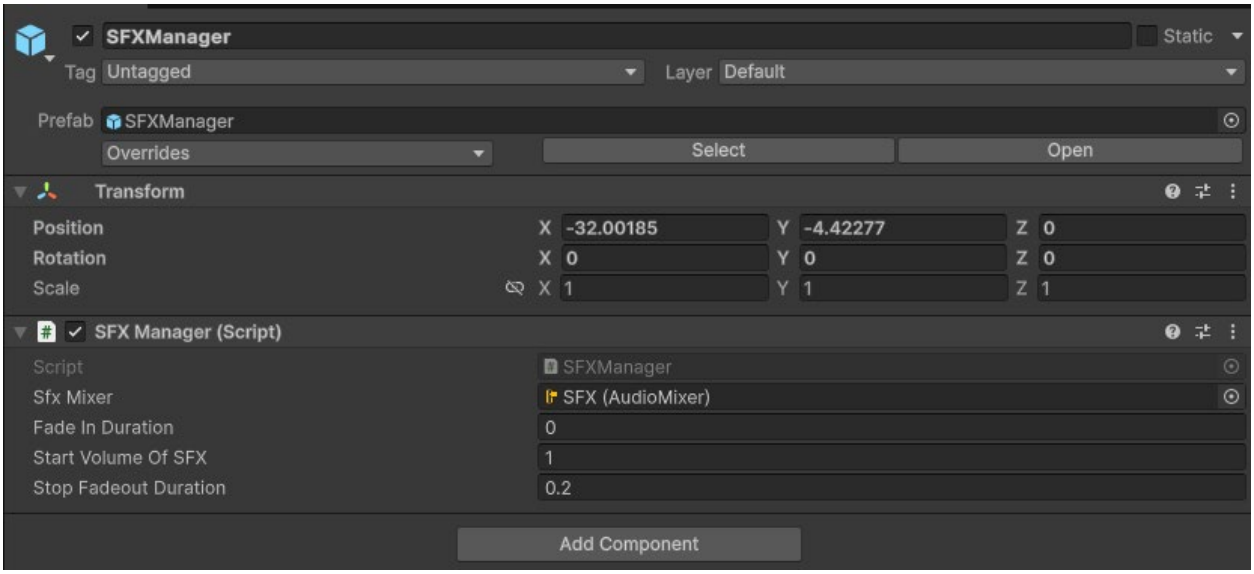


Background Sprites
This list can be filled to use the optional “show next background” function of the action scripts. This function allows you to show the next background of this list by checking a box instead of dragging and dropping a sprite in the action scripts.

The BackGroundManager is a Singleton, so you can also call its methods in your scripts. There are two methods you can call:
SwitchBackGroundTo(sprite)
ShowNextBackground()

SFX Manager

This Manager needs to exist in the scene so you can play sound effects with the tool. For that you can just drag and drop the prefab from the project window in the hierarchy.

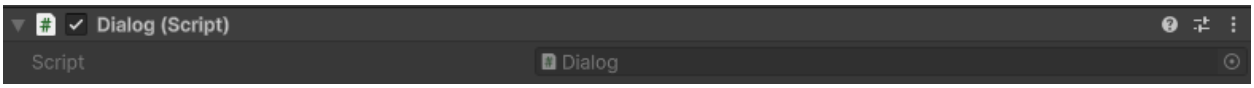


Sfx Mixer: Here you can fill in an “audio mixer group” to adjust the volume of all sounds that are played over the tool
Fade-In Duration: This controls the fade-in duration of the sound effects played over this manager
Start Volume: This controls the start volume of the fade-In.
Stop Fade-Out Duration: This controls the fade-out duration of the sound effects played over this manager

The SFXManager is a Singleton, so you can also call its methods in your scripts.
There are two methods you can call:
PlaySFX(AudioClip)
StopAllSFX()

You can also use your own SFXManager. For that you just need to adjust the code in the ActionPlaySFX script and the dialog script.

Dialog



This script contains the logic of reading and executing the attached actions. Therefore, any game object with this script attached becomes a functional dialog.

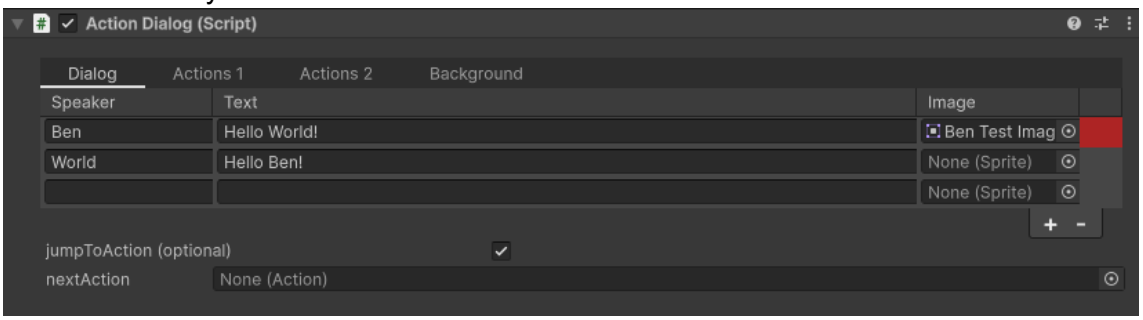
It **must** be the **first component** attached to a dialog prefab! The actions must be attached afterwards.

Actions

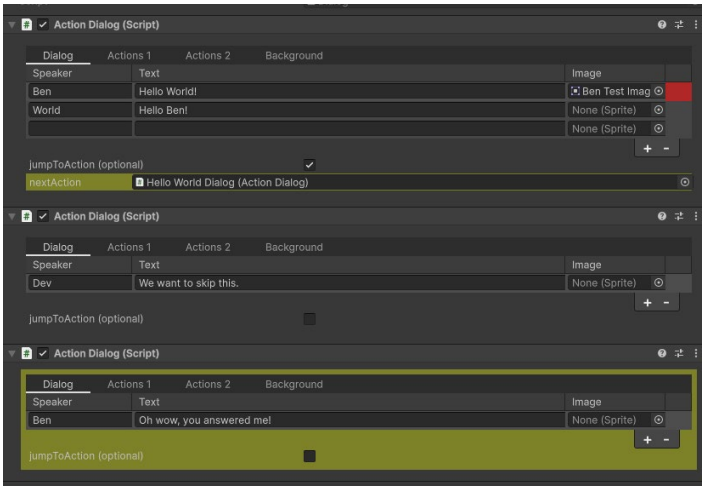
ActionDialog

This action displays linear dialog. It's not limited to one speaker, and you can add as many text lines as you want. You can also trigger animations and audio clips for each text line.
This action consists of 4 Tabs: Dialog, Actions 1, Actions 2 and Background.

<u>Dialog tab</u>
This tab allows you to add text lines and edit its content.

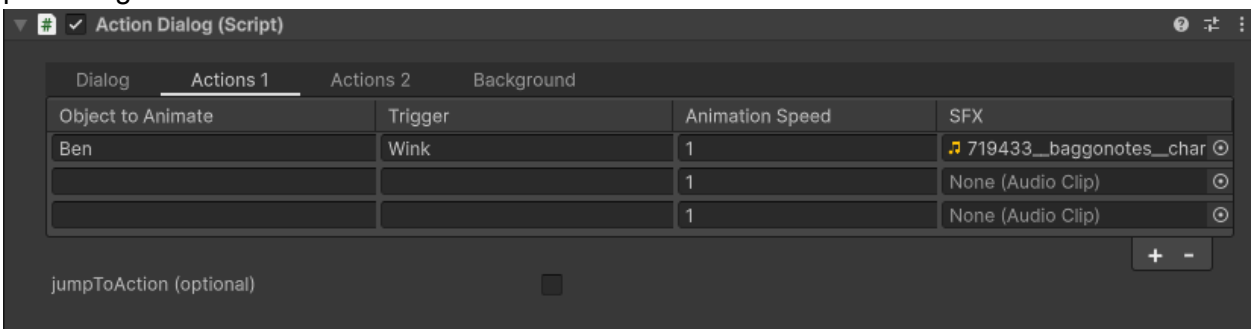


+/-: Add or Remove DialogLines
Speaker: Defines the name of the speaker that will be displayed.
Text: Defines the text that will be displayed.
Image: Defines what speaker image will be shown. If left empty, the previous image will be shown.
Last Column: Indicates whether there are actions defined on the text line or not with a red square.
JumpToAction: Activates and deactivates the nextAction field. When deactivated, it will execute the next action in the prefab hierarchy instead.
NextAction: Defines the action that will be executed afterwards when jumpToAction is activated. Other actions between them will be skipped. When you drag an Action into the nextAction-Field both this field and the affected action will be colored in the same color.



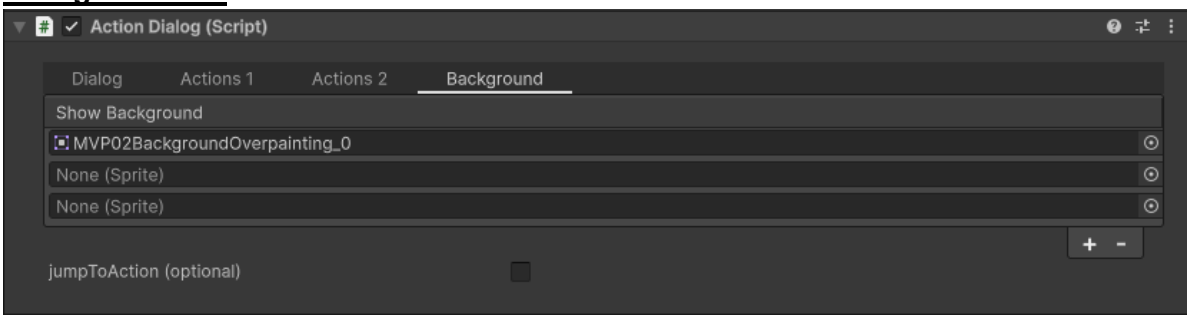
Actions 1/Actions 2 tab

These two tabs allow you to trigger animations or play sounds while a dialog line is showing. Both tabs have the same UI layout, so you can trigger up to 2 animations and 2 sound effects per dialog line.



Object to Animate: The exact name of the game object the animator is attached to.
Trigger: The exact name of the “trigger” in the animator controller.
AnimationSpeed: The speed the animator controller should play the animation with.
SFX: The audio clip that should be played. Drag and drop it from the Project Window

Background tab

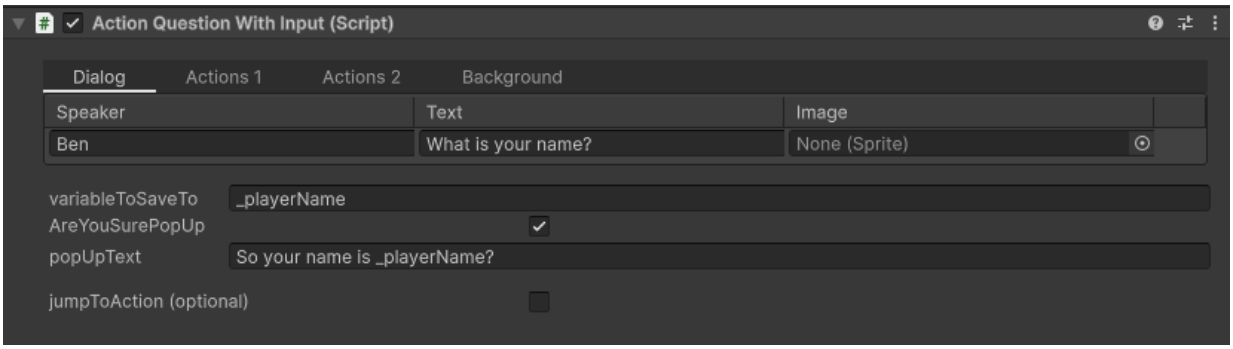


Show Background: Defines if the background should be changed and to what sprite it should be changed. Drag and drop the 2D sprite from the Project Window.

ActionQuestionWithInput

This action displays open questions that must be answered afterwards with keyboard input. It then saves the answer into a variable. You can also trigger animations and audio clips with the question.

This action consists of 4 Tabs: Dialog, Actions 1, Actions 2 and Background. They can be used the same way as in ActionDialog with the difference that you can´t add additional text lines.

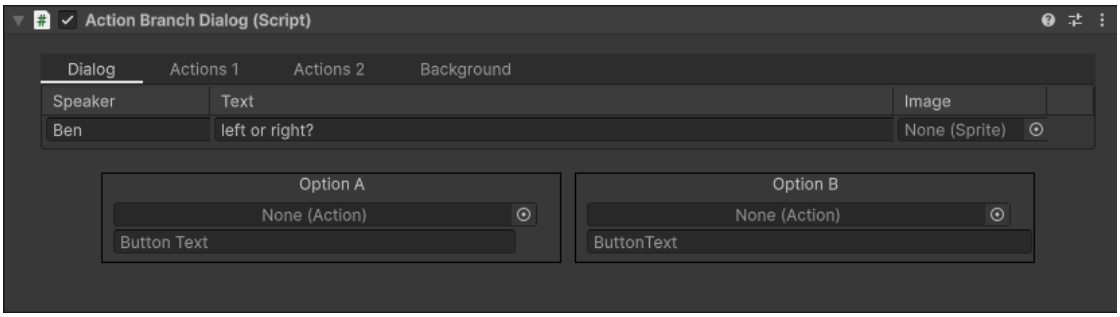


Speaker: Defines the name of the questioner.
Text: Defines the question that will be displayed.
Image: Defines what speaker image will be shown. If left empty, the previous image will be shown.
Last Column: Indicates whether there are actions defined on the text line or not with a red square.
variableToSaveTo: Defines to which variable the answer should be saved. Can only detect variables that were predefined in the DialogVariables Scriptable Object.
AreYouSurePopUp: Activates or deactivates if a confirmation window is shown after the player inputs his answer.
popUpText: Defines the question that is written in the confirmation window. I recommend using the variable name in the question like in the screenshot, so it's more precise

JumpToAction: Activates and deactivates the nextAction field. When deactivated, it will execute the next action in the prefab hierarchy instead.
NextAction: Defines the action that will be executed afterwards when jumpToAction is activated.

ActionBranchDialog

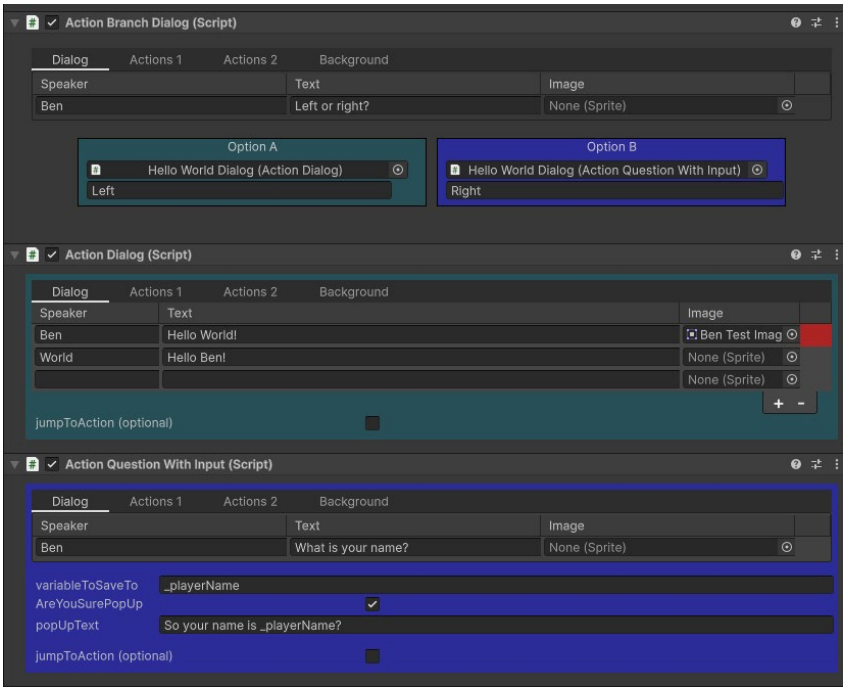
This action displays a question or sentence and two answer choices afterwards. Depending on the choice of the player it can then branch the dialog. You can also trigger animations and audio clips with the question.
This action consists of 4 Tabs: Dialog, Actions 1, Actions 2 and Background. They can be used the same way as in ActionDialog with the difference that you can´t add additional text lines.



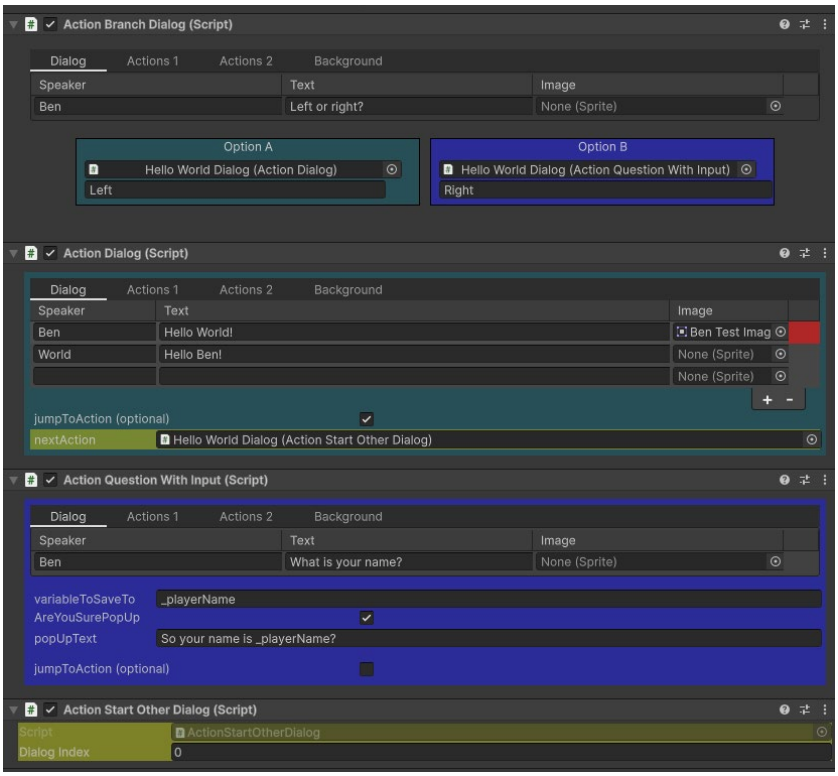
Speaker: Defines the name of the speaker.

Text: Defines the question or sentence that will be displayed.
Image: Defines what speaker image will be shown. If left empty, the previous image will be shown.
Last Column: Indicates whether there are actions defined on the text line or not with a red square.
Option A: Defines what Action should be executed next when the player chooses answer A. The button text defines the text of the answer buttons.
Option B: Defines what Action should be executed next when the player chooses answer B. The button text defines the text of the answer buttons.

When an Action is dragged into one of the Action Fields both the affected Option Field and the dragged in Action are colored in the same color:



Note that this tool is not made for complex branching. However, if you use theBranchDialog-Action together with the jumpToAction-option of the other actions you will be able to do some light branching.

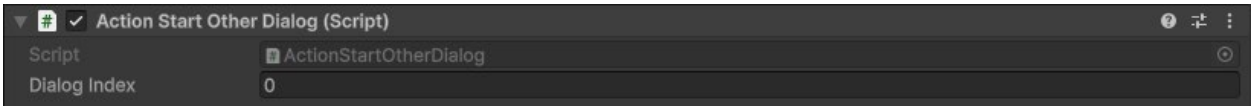


ActionEndDialog

This action ends the dialog instantly. Without this action the dialog ends automatically after the last action was reached and processed. So, you only need this for branching.

ActionStartOtherDialog

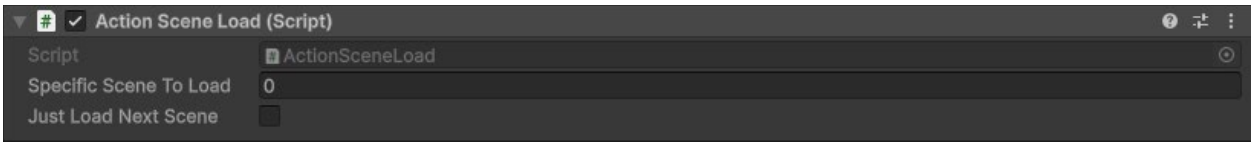
This action ends the dialog instantly and starts a new dialog.



Dialog Index: Represents the position in the dialog list of the Dialog Manager. The dialog found at that position will start. The list starts with Position 0.

ActionSceneLoad

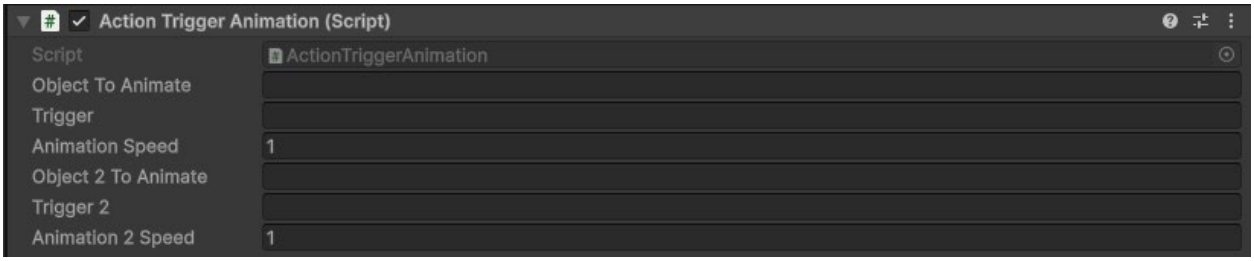
This action loads a new scene.



Specific Scene: This Index represents the position in the scene list of the build profile. The scene found at that position will load. The list starts with Position 0.
Load Next Scene: When checked, it just loads the scene at the next position of the scene list instead.

ActionTriggerAnimation

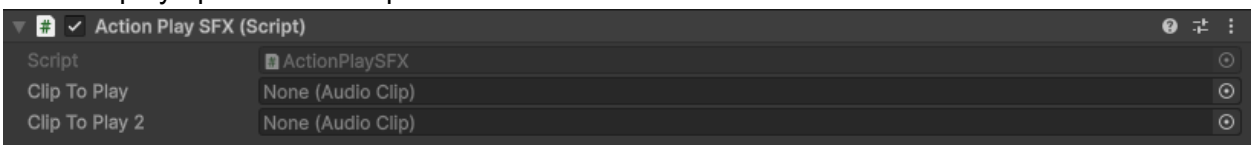
Like the “Action 1/Action 2” tabs in ActionDialog this action triggers animations.
In contrast to the tabs this action isn’t displaying or changing any dialog text.
You can trigger up to 2 animations.



Object to Animate: The exact name of the game object the animator is attached to.
Trigger: The exact name of the “trigger” in the animator controller.
AnimationSpeed: The speed the animator controller should play the animation with.

ActionPlaySFX

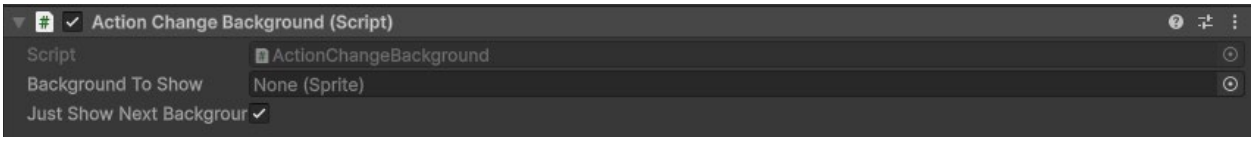
Like the “Action 1/Action 2” tabs in ActionDialog this action plays sound effects.
In contrast to the tabs this action isn’t displaying or changing any dialog text.
You can play up to 2 audio clips.



ClipToPlay: The audio clip that should be played. Drag and drop it from the Project Window.

ActionChangeBackground

Like the “Background” tab in ActionDialog this action changes the background.
In contrast to the “Background” tab this action isn’t displaying or changing any dialog text.

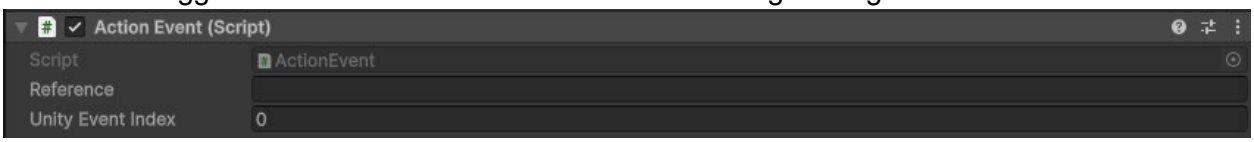


Background To Show: The 2D sprite the background should be changed to. Drag and drop it from the Project Window.
Show Next Background: Changes the background to the next sprite in the list of the BackgroundManager.

You **need** to choose one of these two options for the whole scene! Both options at the same time will not work.

ActionEvent

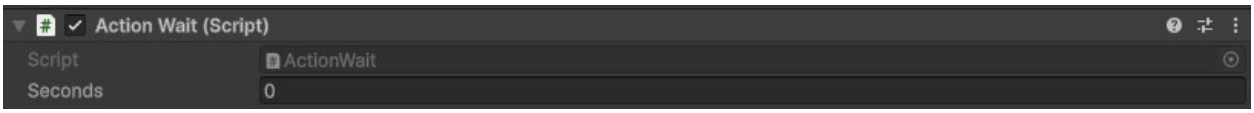
This action triggers an event from the event list of the Dialog Manager.



Reference: You can name the event you are triggering here. It helps you to keep the overview, when the dialog gets bigger.
Unity Event Index: Triggers the event that's positioned at the chosen index in the Unity Event List of the DialogManager. The list starts with Position 0.

ActionWait

This action delays the processing of the next action. Therefore, it is perfect for cutscenes or similar situations where you don't want the next dialog to be displayed before the animations are finished.



Seconds: Determines how long the dialog should wait before processing the next action. It takes a decimal number.

How to create your own custom Action

If you want to create a new action you need to do the following:

1. Create a new script and name it Action… + the thing it does. "ActionPlayVideo" for example
2. Make sure it inherits from "Action". This is the base class that all action scripts have to inherit from.
3. Open the "Dialog" script and find the ExecuteActions()-method in it.

```
8 Verweise
public void ExecuteActions()
{
    if (firstTime)
    {
        dM.ShowDialogPanel();
        dM.ShowContinueButton();
        firstTime = false;
    }

    if (actionArray[actionCount] is ActionDialog dialog)
    {
        HandleNormalDialog(dialog);
    }
    else if (actionArray[actionCount] is ActionQuestionWithInput question)
    {
        HandleInputQuestion(question);
    }
    else if (actionArray[actionCount] is ActionBranchDialog branch)
    {
        HandleBranchingOfDialog(branch);
    }
    else if (actionArray[actionCount] is ActionChangeBackground backgroundChanger)
    {
        backgroundChanger.SwitchBackground();
        ExecuteNextLineOrActionOrReportDialogAsProcessed(backgroundChanger);
    }
    else if (actionArray[actionCount] is ActionPlaySFX sfxPlayer)
    {
        sfxPlayer.PlaySFX();
        ExecuteNextLineOrActionOrReportDialogAsProcessed(sfxPlayer);
    }
    else if (actionArray[actionCount] is ActionTriggerAnimation animationPlayer)
    {
        animationPlayer.TriggerAnimation();
        ExecuteNextLineOrActionOrReportDialogAsProcessed(animationPlayer);
    }
    else if (actionArray[actionCount] is ActionSceneLoad sceneLoader)
    {
        sceneLoader.LoadScene();
    }
    else if (actionArray[actionCount] is ActionEndDialog dialogEnder)
    {
        dM.DialogProcessed();
    }
    else if (actionArray[actionCount] is ActionStartOtherDialog otherDialogStarter)
    {
        dM.DialogProcessed();
        dM.StartDialogNumber(otherDialogStarter.dialogIndex);
    }
    else if (actionArray[actionCount] is ActionEvent unityEventAction)
    {
        unityEventAction.InvokeEvent();
        ExecuteNextLineOrActionOrReportDialogAsProcessed(unityEventAction);
    }
    else if (actionArray[actionCount] is ActionWait waiter)
    {
        if (isWaiting)
        {
            return;
        }
        isWaiting = true;
        StartCoroutine(WaitCoroutine(waiter));
    }


}
```

4. Now add another else-if statement that checks for your created class. Type in it what should happen. Note that you must always **end** the statement **with** the following method call: **ExecuteNextLineOrActionOrReportDialogAsProcessed()!**

```
else if (actionArray[actionCount] is ActionPlayVideo videoPlayer)
{
    //here you can define what should happen

    //always end with the following or dM.dialogProcessed() if needed
    ExecuteNextLineOrActionOrReportDialogAsProcessed(videoPlayer);
}
```

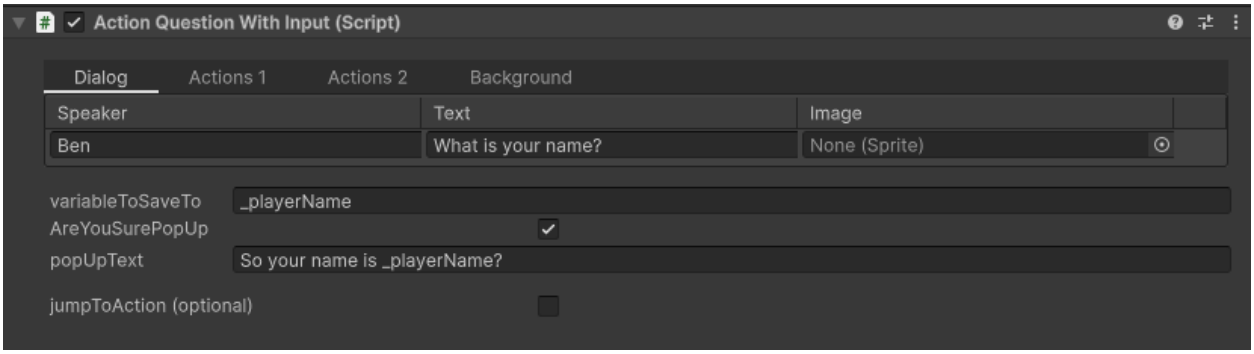
Alternatively, you can report the dialog as processed with dM.DialogProcessed() if your action should also end the dialog instantly.

4. How to use variables

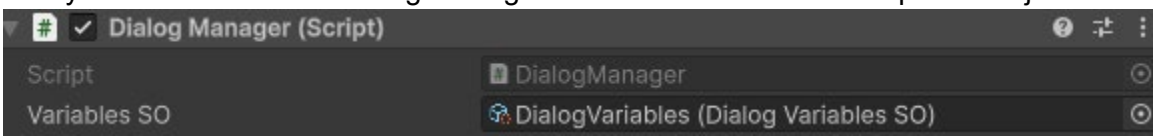
To use variables in the tool you first need to predefine them in the DialogVariables-Scriptable Object. For that you can just edit the existing ones or extend the list and add new variables.

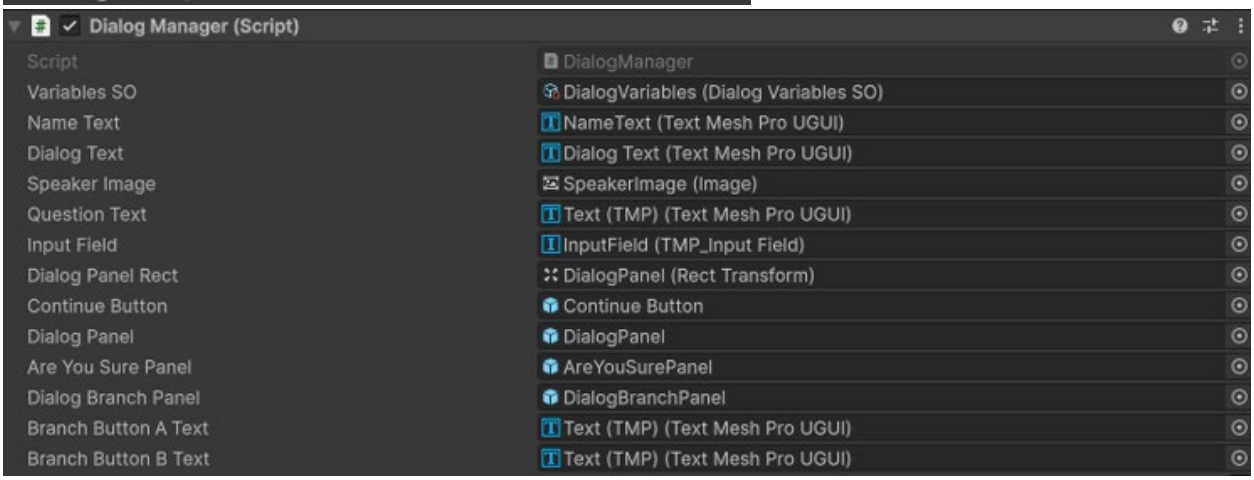
You **need** to use an **underscore _ at the beginning** of your variable names!
With that underscore the tool can automatically detect variables within your text, as long as you also use the **underscore_** there.

So, for example, if you write “Hey _playerName!” in one of your actions, the displayed text would be automatically transformed to “Hey Ben!”, if your players name is Ben.
Here is another example:



Lastly make sure that the Dialog Manager has a reference to the Scriptable Object:



5. How to change the UI of the Dialog

For visual changes of the UI, you can edit or swap out the referenced UI-Elements that are listed in the Canvas of the Dialog Manager prefab.